

1. feladat – Egyszerű jegyzetalkalmazás (szöveg + nagybetűs konverter)

Cél: Fejlesszen egy egyszerű MAUI oldalt, ahol a felhasználó szöveget írhat be, elmentheti azt, és a mentett szöveg nagybetűs formában jelenik meg. A felhasználónak lehetősége legyen a tárolt szöveg törlésére is.

Követelmények:

- **View (MainPage):**
 - egy Entry vezérlő szövegbevitelhez,
 - egy Save gomb,
 - egy Clear gomb,
 - egy Label, amely a mentett szöveget nagybetűs formában jeleníti meg.
- **ViewModel (MainViewModel, a ViewModelBase osztályból származtatva):**
 - Input : string – a beviteli mezőhöz kötött tulajdonság,
 - NoteText : string – a mentett szöveget megjelenítő Label tulajdonságához kötve,
 - Save() metódus – beállítja: NoteText = Input,
 - Clear() metódus – az Input és NoteText értékét is üresre állítja.
- **Helper (UppercaseConverter):**
 - implementálja az IValueConverter interfészt,
 - a bemeneti szöveget nagybetűs formára alakítja.
- **Interakció:**
 - a Save és Clear gomboknak a ViewModel Save() és Clear() metódusait kell meghívniuk a code-behind fájlból, gombkattintási eseményeken keresztül.

2. feladat – Todo lista (ObservableCollection + törlés)

Cél: Valósítson meg egy egyszerű Todo lista alkalmazást, amelyben a felhasználó új feladatokat adhat hozzá, listában jelenítheti meg őket, készre jelölheti az egyes feladatokat, valamint törölheti a kijelölt elemet.

Követelmények:

- **View (TodoPage):**
 - egy Entry új feladat címének megadásához,

- egy Add gomb,
- egy CollectionView, amely a feladatokat jeleníti meg; minden elem tartalmazzon egy CheckBox vezérlőt és egy Label vezérlőt,
- egy Remove selected gomb, amely csak akkor legyen engedélyezett, ha van kijelölt elem.
- **Model (TodoItem, POCO):**
 - Title : string
 - IsDone : bool
- **ViewModel-ek:**
 - TodoItemViewModel – a TodoItem burkolóosztálya, amely implementálja az INotifyPropertyChanged interfészt. Tulajdonságai: Title, IsDone.
 - TodoViewModel:
 - Items : ObservableCollection<TodoItemViewModel> – a feladatok listája,
 - NewTitle : string – az új feladat címeként megadott szöveg,
 - Selected : TodoItemViewModel? – az aktuálisan kijelölt feladat,
 - Add() – új elemet ad a listához,
 - Remove() – eltávolítja a kijelölt elemet.
- **Helper:**
 - NullToBoolConverter – true értéket ad vissza, ha a kötött objektum nem null, egyébként false értéket. A Remove gomb engedélyezéséhez és letiltásához használja.

3. feladat – Hőmérséklet-konverter (Celsius ↔ Fahrenheit)

Cél: Készítsen hőmérséklet-konvertert két beviteli mezővel. Ha a felhasználó az egyik mezőben Celsius- vagy Fahrenheit-értéket ad meg, a másik mező automatikusan frissüljön. Egy Reset gomb állítsa vissza az alapértelmezett kiindulási értékeket.

Követelmények:

- **View (TempPage):**
 - egy Entry a Celsius-érték megadásához,
 - egy Entry a Fahrenheit-érték megadásához,

- egy Reset gomb, amely csak akkor legyen engedélyezett, ha az aktuális értékek eltérnek az alapértékektől: 0 °C / 32 °F.
 - **ViewModel (TempViewModel):**
 - Celsius : double – a setter frissítse a Fahrenheit-értéket,
 - Fahrenheit : double – a setter frissítse a Celsius-értéket,
 - privát _isUpdating flag a rekurzív frissítések megakadályozására,
 - CanReset : bool – true, ha az értékek eltérnek az alapértékektől,
 - Reset() – visszaállítja az értékeket 0 °C és 32 °F értékre.
 - **Helper (DoubleFormatConverter):**
 - biztosítsa, hogy a numerikus értékek egy tizedesjeggyel jelenjenek meg.
 - **Interakció:**
 - a Reset gomb IsEnabled tulajdonsága legyen a ViewModel CanReset tulajdonságához kötve.
-

4. feladat – Terméklista kereséssel (szűrés + PriceConverter)

Cél: Készítsen egy terméklista oldalt, ahol a felhasználó termékekre kereshet. A lista valós időben frissüljön a keresési kifejezés alapján, az árak pedig formázott pénznemként jelenjenek meg.

Követelmények:

- **View (ProductListPage):**
 - egy SearchBar a keresési kifejezés megadásához,
 - egy CollectionView, amely a szűrt termékeket jeleníti meg névvel és formázott árral.
- **Model (Product, POCO):**
 - Name : string
 - Price : decimal
- **ViewModel-ek:**
 - ProductItemViewModel – a Product burkolóosztálya, amely implementálja az INotifyPropertyChanged interfészt, és elérhetővé teszi a Name és Price tulajdonságokat.
 - ProductListViewModel:

- tartalmazza az összes terméket egy privát gyűjteményben,
 - elérhetővé teszi az `Items` : `ObservableCollection<ProductItemViewModel>` gyűjteményt,
 - implementálja a `Filter(string query)` metódust, amely az `Items` tartalmát csak a keresési feltételnek megfelelő termékekkel frissíti.
- **Helper (PriceConverter):**
 - implementálja az `IValueConverter` interfészt,
 - a decimal értékeket lokalizált pénznemformátumú szöveggé alakítja.
 - **További követelmény – debounce:**
 - amikor a keresőmező szövege megváltozik, a `Filter()` metódust ne hívja meg azonnal,
 - ehelyett valósítson meg 300 ms késleltetést `DispatcherTimer` (`IDispatcherTimer` MAUI-ban) használatával,
 - a szűrés csak akkor fusson le, ha a késleltetési idő alatt nem érkezik újabb bevétel.

5. feladat – Többnézetes alkalmazás Shell használatával (Feladatok + Statisztika)

Cél: Fejlesszen többnézetes alkalmazást Shell navigáció használatával. Az alkalmazás két lapot tartalmazzon:

1. **Tasks** – ahol a felhasználó feladatokat adhat hozzá és kezelhet,
2. **Stats** – ahol az összesített statisztikák jelennek meg, például az összes feladat és a befejezett feladatok száma.

Mindkét nézet ugyanazt a mögöttes adatforrást használja.

Követelmények:

- **Shell:**
 - az `AppShell.xaml` definiáljon két lapot: `Tasks` és `Stats`.
- **Model (TaskItem, POCO):**
 - `Title` : string
 - `Done` : bool

- **ViewModel-ek:**

- TaskItemViewModel – a TaskItem burkolóosztálya, amely implementálja az INotifyPropertyChanged interfészt. Tulajdonságai: Title, Done.
- TaskStore – statikus osztály, amely közös ObservableCollection<TaskItemViewModel> gyűjteményt biztosít mindkét nézet számára.
- TasksViewModel:
 - Items – a TaskStore.Items gyűjteményhez kötve,
 - NewTitle : string – az új feladat neve,
 - Add() – új feladatot ad a gyűjteményhez,
 - Toggle(TaskItemViewModel) – átváltja egy feladat befejezett állapotát,
 - Total : int – az összes feladat száma,
 - Completed : int – a befejezett feladatok száma,
 - fel kell iratkoznia a gyűjtemény és az egyes elemek tulajdonságváltozásaira annak érdekében, hogy a Total és Completed értékek mindig naprakészek legyenek.
- StatsViewModel:
 - szintén figyeli a TaskStore.Items gyűjteményt,
 - elérhetővé teszi a Total és Completed tulajdonságokat,
 - automatikusan frissül, amikor a feladatlista megváltozik.

- **Helper (BoolToTextDecorationConverter):**

- a Done logikai értéket TextDecorations értéké alakítja,
- true esetén áthúzott szöveg jelenjen meg.

- **View-k:**

- **TasksPage:**
 - egy Entry és egy Add gomb új feladatok létrehozásához,
 - egy CollectionView, amely megjeleníti a feladatokat; minden elem tartalmazzon egy CheckBox vezérlőt és a feladat címét,
 - Label vezérlők a Total és Completed értékek megjelenítésére.

- **StatsPage:**
 - Label vezérlők az összes feladat és a befejezett feladatok számának megjelenítésére, a StatsViewModel tulajdonságaihoz kötve.